

เอกสารโครงสร้างฐานข้อมูล — ระบบมหาวิทยาลัย (Multi-Schema)

เอกสารนี้อธิบายโครงสร้างทั้งหมดของ `university_multi_schema.sql` ซึ่งเป็นฐานข้อมูล PostgreSQL ที่แบ่งตารางออกเป็น **7 schema** ตามระบบย่อยของมหาวิทยาลัย เพื่อให้ดู ERD แยกโซนได้ชัดเจนใน pgAdmin (คลิกขวาที่ schema → Generate ERD)

ภาพรวมทั้งระบบ

Schema	ชื่อระบบ	จำนวนตาราง	เชื่อมต่อ (FK ออกไปหา)
core	ตารางแกนกลาง	10	— (เป็นฐานให้ทุก schema)
u1_reg	ทะเบียนนักศึกษา	5	core
u2_enroll	ลงทะเบียนเรียน	1	core, u1_reg
u3_grade	ผลการเรียน	4	u1_reg, u2_enroll
u4_fee	ค่าเทอม	4	core, u2_enroll
u6_hr	บุคลากร	32	core
u5_lib	ห้องสมุด	11	core, u6_hr
รวม		67 ตาราง	

ลำดับการสร้าง schema ใน SQL คือ `core` → `u1_reg` → `u2_enroll` → `u3_grade` → `u4_fee` → `u5_lib` → `u6_hr` แต่ในทางความสัมพันธ์จริง `u6_hr` ต้องถูกสร้างก่อน `u5_lib` เพราะ `u5_lib.members` มี FK ไปหา `u6_hr.faculty_members` และ `u6_hr.staff`

1. Schema `core` — ตารางแกนกลาง (10 ตาราง)

ตารางฐานที่ทุกระบบย่อยอ้างอิงร่วมกัน (สถาบัน, คณะ, สาขา, รายวิชา, ภาคการศึกษา, อาคาร/ห้อง, นักศึกษา, อาจารย์)

ตาราง	Primary Key	คอลัมน์หลัก	Foreign Key
hr_institutes	institute_id (serial)	institute_code (unique), institute_name, address, established_date	—
faculties	faculty_id (varchar 5)	faculty_name, institute_id	→ hr_institutes.institute_id (ON DELETE SET NULL)
departments	department_id (varchar 5)	department_name, faculty_id (NOT NULL)	→ faculties.faculty_id (ON DELETE CASCADE)
majors	major_id (varchar 5)	major_name, department_id (NOT NULL)	→ departments.department_id (ON DELETE CASCADE)
courses	course_id (varchar 10)	course_name, credits, department_id	→ departments.department_id (ON DELETE SET NULL)
semesters	semester_id (serial)	academic_year, term, start_date, end_date	—
buildings	building_id (varchar 5)	building_name	—
rooms	room_id (varchar 10)	building_id (NOT NULL), capacity	→ buildings.building_id (ON DELETE CASCADE)
students	student_id (varchar 10)	first_name, last_name, faculty_id, major_id, enrollment_year	→ faculties.faculty_id, → majors.major_id (ทั้งคู่ ON DELETE SET NULL)
instructors	instructor_id (varchar 10)	first_name, last_name, department_id, email (unique)	→ departments.department_id (ON DELETE SET NULL)

ข้อสังเกต: ลำดับชั้นคือ สถาบัน → คณะ → สาขา (department) → วิชาเอก (major) เป็นโครงสร้างแบบต้นไม้ 4 ชั้น ส่วนรายวิชา (courses) ผูกกับ department ไม่ใช่ major

2. Schema `u1_reg` — ทะเบียนนักศึกษา (5 ตาราง)

ตาราง	Primary Key	คอลัมน์หลัก	Foreign Key
<code>prerequisites</code>	(<code>course_id</code> , <code>prerequisite_course_id</code>) composite	—	ทั้งสองคอลัมน์ → <code>core.courses.course_id</code> (CASCADE)
<code>sections</code> (กลุ่มเรียน)	<code>section_id</code> (serial)	<code>course_id</code> , <code>semester_id</code> , <code>instructor_id</code> , <code>room_id</code> , <code>schedule_time</code> , <code>max_seats</code>	→ <code>core.courses</code> , → <code>core.semesters</code> (CASCADE); → <code>core.instructors</code> , → <code>core.rooms</code> (SET NULL)
<code>advisors</code> (อาจารย์ที่ปรึกษา)	<code>student_id</code>	<code>instructor_id</code>	→ <code>core.students</code> , → <code>core.instructors</code> (CASCADE)
<code>scholarships</code>	<code>scholarship_id</code> (serial)	<code>scholarship_name</code> , <code>fund_amount</code>	—
<code>student_scholarships</code>	(<code>student_id</code> , <code>scholarship_id</code> , <code>semester_id</code>) composite	<code>awarded_date</code> (default วันนี้)	→ <code>core.students</code> , → <code>u1_reg.scholarships</code> , → <code>core.semesters</code> (ทั้งหมด CASCADE)

ข้อสังเกตสำคัญ: ตาราง `advisors` ใช้ `student_id` เป็น PK เดี่ยว (ไม่รวม `semester_id`) หมายความว่า **นักศึกษา 1 คนมีอาจารย์ที่ปรึกษาได้เพียง 1 คนตลอดการศึกษา** ไม่สามารถเปลี่ยนอาจารย์ที่ปรึกษาต่อภาคการศึกษาได้ในโครงสร้างปัจจุบัน หากต้องการรองรับการเปลี่ยนอาจารย์ที่ปรึกษาในอนาคต ควรเพิ่ม `semester_id` หรือ `effective_date` เข้าไปใน PK

3. Schema `u2_enroll` — ลงทะเบียนเรียน (1 ตาราง)

ตาราง	Primary Key	คอลัมน์หลัก	Foreign Key
<code>enrollments</code>	<code>enrollment_id</code> (serial)	<code>student_id</code> , <code>section_id</code> , <code>course_id</code> , <code>enrollment_date</code> , <code>enrollment_status</code> (default <code>ENROLLED</code>), <code>midterm_score</code> , <code>final_score</code> , <code>total_score</code> (generated column), <code>letter_grade</code> , <code>grade_point</code>	→ <code>core.students</code> , → <code>u1_reg.sections</code> , → <code>core.courses</code> (ทั้งหมด CASCADE)

คอลัมน์คำนวณอัตโนมัติ: `total_score` เป็น `GENERATED ALWAYS AS (COALESCE(midterm_score,0) + COALESCE(final_score,0)) STORED` — คำนวณจาก midterm + final อัตโนมัติ ไม่ต้องเขียนค่าลงไปตรง ๆ (และเขียนทับไม่ได้)

ข้อสังเกต: ตารางนี้เก็บ midterm/final/total/letter_grade/grade_point ไว้ในตัวเอง ในขณะที่ schema `u3_grade` แยกไปเก็บรายละเอียดระดับ assignment/การเข้าเรียน/ประวัติการเก้เกรด กล่าวคือ `u2_enroll.enrollments = คะแนนสรุปและเกรดตัดจริง`, `u3_grade = รายละเอียดย่อยที่ประกอบขึ้นเป็นคะแนนนั้น` ไม่ใช่ข้อมูลซ้ำซ้อนกันโดยตรง แต่ผู้ดูแลระบบต้องระวังเรื่อง sync ระหว่างสองจุดนี้เอง (ไม่มี trigger คำนวณ midterm/final จาก `student_scores` ให้อัตโนมัติ)

4. Schema `u3_grade` — ผลการเรียน (4 ตาราง)

ตาราง	Primary Key	คอลัมน์หลัก	Foreign Key
<code>assignments</code>	<code>assignment_id</code> (serial)	<code>section_id</code> , <code>title</code> , <code>max_score</code> , <code>due_date</code>	→ <code>u1_reg.sections</code> (CASCADE)
<code>student_scores</code>	<code>score_id</code> (serial)	<code>enrollment_id</code> , <code>assignment_id</code> , <code>score_obtained</code>	→ <code>u2_enroll.enrollments</code> , → <code>u3_grade.assignments</code> (CASCADE)
<code>attendance</code>	<code>attendance_id</code> (serial)	<code>enrollment_id</code> , <code>class_date</code> , <code>status</code>	→ <code>u2_enroll.enrollments</code> (CASCADE)
<code>grade_change_logs</code>	<code>log_id</code> (serial)	<code>enrollment_id</code> , <code>old_grade</code> , <code>new_grade</code> , <code>changed_by</code> , <code>reason</code> , <code>changed_at</code>	→ <code>u2_enroll.enrollments</code> (CASCADE)

ข้อสังเกต: `grade_change_logs.changed_by` เป็น `character varying(100)` ธรรมดา ไม่ได้ FK ไปหาอาจารย์/เจ้าหน้าที่คนใดจริง ๆ (เก็บเป็นชื่อ/รหัสข้อความอิสระ) — หากต้องการตรวจสอบย้อนกลับว่าใครแก้เกรด ควรพิจารณาผูกกับ `u6_hr.faculty_members` หรือ `u6_hr.staff` ในอนาคต

5. Schema `u4_fee` — ค่าเทอม (4 ตาราง)

ตาราง	Primary Key	คอลัมน์หลัก	Foreign Key
<code>fee_types</code>	<code>fee_type_id</code> (serial)	<code>fee_name</code> , <code>description</code>	—
<code>invoices</code>	<code>invoice_id</code> (serial)	<code>student_id</code> , <code>semester_id</code> , <code>total_amount</code> , <code>issue_date</code> , <code>due_date</code> , <code>is_paid</code>	→ <code>core.students</code> , → <code>core.semesters</code> (CASCADE)
<code>invoice_items</code>	<code>item_id</code> (serial)	<code>invoice_id</code> , <code>fee_type_id</code> , <code>enrollment_id</code> (nullable), <code>amount</code> , <code>description</code>	→ <code>u4_fee.invoices</code> , → <code>u4_fee.fee_types</code> (CASCADE); → <code>u2_enroll.enrollments</code> (SET NULL)
<code>payments</code>	<code>payment_id</code> (serial)	<code>invoice_id</code> , <code>amount_paid</code> , <code>payment_date</code> , <code>payment_method</code>	→ <code>u4_fee.invoices</code> (CASCADE)

ข้อสังเกต: `invoices.total_amount` เป็นค่าที่กรอกเองแยกต่างหาก ไม่ได้คำนวณจาก `SUM(invoice_items.amount)` อัตโนมัติ — ต้องดูแลให้ตรงกันเองในระดับแอปพลิเคชัน เช่นเดียวกับยอดชำระใน `payments` ที่ไม่มีการตรวจสอบว่ารวมแล้วเท่ากับ `invoices.total_amount` หรือไม่

6. Schema `u6_hr` — บุคลากร (32 ตาราง)

Schema ที่ใหญ่ที่สุด แบ่งได้เป็นกลุ่มย่อยดังนี้

6.1 ตารางฐานข้อมูลอ้างอิง (lookup)

ตาราง	คอลัมน์หลัก
academic_ranks	rank_name, abbreviation (เช่น อ., ผศ., รศ., ศ.)
employment_types	type_name, description
positions	position_title, description, salary_level
leave_types	leave_name, max_days_per_year
certifications	cert_name, issuing_org
committees	committee_name, description
internal_trainings	topic_name, organizer, training_date

6.2 ตารางบุคคลหลัก

ตาราง	Primary Key	Foreign Key
faculty_members (อาจารย์)	faculty_member_id (serial)	department_id → core.departments (SET NULL); rank_id → academic_ranks ; employment_type_id → employment_types ; instructor_id → core.instructors (เชื่อม HR เข้ากับทะเบียนการสอน, SET NULL)
staff (บุคลากรสายสนับสนุน)	staff_id (serial)	department_id → core.departments ; position_id → positions ; employment_type_id → employment_types (ทั้งหมด SET NULL)

6.3 ตารางที่ใช้รูปแบบ "Polymorphic Association" (owner_type + owner_id)

ตารางกลุ่มนี้ใช้คู่คอลัมน์ owner_type (ข้อความ เช่น 'faculty_member' หรือ 'staff') + owner_id (เลขอ้างอิง) แทนการทำ FK ตรง ๆ เพื่อให้ตารางเดียวรองรับได้ทั้งอาจารย์และบุคลากรสายสนับสนุน **ข้อควรระวัง: PostgreSQL ไม่มีกลไกบังคับ referential integrity ให้รูปแบบนี้โดยอัตโนมัติ** (ไม่มี FK constraint จริงในตารางเหล่านี้) การตรวจสอบว่า owner_id มีอยู่จริงในตาราง faculty_members หรือ staff ตาม owner_type ต้องทำที่ชั้นแอปพลิเคชันเอง

รายชื่อตารางที่ใช้ pattern นี้: employee_dependents , employee_addresses , employee_documents , asset_allocations , attendance_logs , performance_appraisals , employee_certifications , leave_requests , salaries , salary_increments , payroll_details , committee_members , employee_trainings , exit_interviews , room_bookings , emergency_contacts

ตาราง	คอลัมน์เฉพาะ	FK จริงที่มี
employee_dependents	first_name, last_name, relationship, birth_date, education_level, is_eligible_benefit	—
employee_addresses	address_line1, city, province, postal_code, country (default Thailand)	—
employee_documents	document_type, file_path, uploaded_at	—
asset_allocations	assigned_date, return_date	asset_id → assets (CASCADE)
attendance_logs	check_in, check_out, status (default Present)	—
performance_appraisals	evaluator_id (ไม่มี FK), fiscal_year, score, comments	—
employee_certifications	issue_date, expire_date	certification_id → certifications (CASCADE)
leave_requests	start_date, end_date, reason, status (default Pending)	leave_type_id → leave_types (SET NULL)
leave_approvals	approver_id (ไม่มี FK), approval_status, approval_date	request_id → leave_requests (CASCADE)
salaries	base_salary, effective_date	—
salary_increments	previous_salary, new_salary, increment_date, reason	—
payroll_details	earnings, deductions, net_pay	payroll_run_id → payroll_runs (CASCADE)
committee_members	role_in_committee (default Member)	committee_id → committees (CASCADE)
employee_trainings	status (default Completed)	training_id → internal_trainings (CASCADE)
exit_interviews	resignation_date, reason, feedback	—
room_bookings	room_number (varchar ธรรมดา, ไม่ใช่ FK), booking_date, start_time, end_time, purpose	—
emergency_contacts	contact_name, relationship, phone_number	—

ข้อสังเกต: u6_hr.room_bookings เก็บ room_number เป็นข้อความอิสระ ไม่ได้ผูกกับ core.rooms ต่างจาก u5_lib.room_reservations ที่ผูกกับ u5_lib.meeting_rooms อย่างถูกต้องด้วย FK จริง — ถ้าตั้งใจให้ room_bookings ใช้ห้องชุดเดียวกับ core.rooms ควรแก้เป็น FK เพื่อกันข้อมูลพิมพ์ผิด/ห้องไม่มีอยู่จริง

6.4 ตารางอื่น ๆ ที่มี FK ปกติ

ตาราง	Foreign Key
assets	— (ไม่มี FK ขาเข้า)
payroll_runs	—
curricula	department_id → core.departments (SET NULL)
publications (ผลงานตีพิมพ์)	faculty_member_id → faculty_members (CASCADE)
research_grants (ทุนวิจัย)	faculty_member_id → faculty_members (SET NULL)
department_budgets (งบประมาณสาขา)	department_id → core.departments (CASCADE)

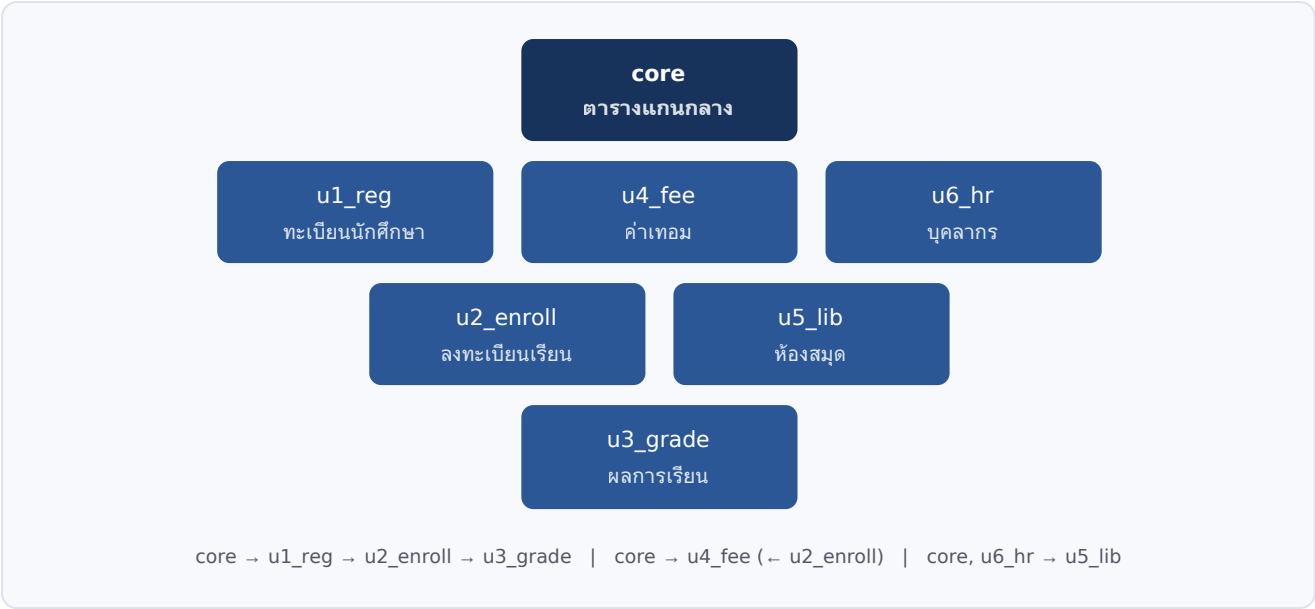
7. Schema `u5_lib` — ห้องสมุด (11 ตาราง)

ตาราง	Primary Key	คอลัมน์หลัก	Foreign Key
<code>media_types</code>	<code>media_type_id</code> (varchar 10)	<code>type_name</code>	—
<code>categories</code>	<code>category_id</code> (varchar 10)	<code>dewey_code</code> , <code>category_name</code>	—
<code>resources</code> (รายการทรัพยากร)	<code>resource_id</code> (varchar 20)	<code>title</code> , <code>author</code> , <code>publisher</code> , <code>publish_year</code> , <code>isbn</code>	→ <code>categories</code> , → <code>media_types</code> (SET NULL)
<code>resource_items</code> (สำเนาจริงแต่ละเล่ม)	<code>item_id</code> (varchar 20)	<code>location_zone</code> , <code>status</code> (default Available)	→ <code>resources</code> (CASCADE)
<code>digital_resources</code>	<code>digital_id</code> (varchar 20)	<code>file_path_url</code> , <code>file_format</code> , <code>access_level</code>	→ <code>resources</code> (CASCADE)
<code>members</code> (สมาชิกห้องสมุด)	<code>member_id</code> (varchar 20)	<code>first_name</code> , <code>last_name</code> , <code>email</code> (unique), <code>phone</code> , <code>member_type</code> , <code>faculty_department</code>	→ <code>core.students</code> (<code>student_id</code>), → <code>u6_hr.faculty_members</code> (<code>hr_faculty_member_id</code>), → <code>u6_hr.staff</code> (<code>hr_staff_id</code>) — ทั้งหมด SET NULL
<code>borrow_transactions</code>	<code>transaction_id</code> (varchar 20)	<code>borrow_date</code> , <code>due_date</code> , <code>return_date</code> , <code>status</code>	→ <code>members</code> , → <code>resource_items</code> (SET NULL)
<code>reservations</code> (จองหนังสือ)	<code>reservation_id</code> (varchar 20)	<code>reserve_date</code> , <code>status</code> (default Pending)	→ <code>members</code> , → <code>resources</code> (SET NULL)
<code>fines</code> (ค่าปรับ)	<code>fine_id</code> (varchar 20)	<code>fine_amount</code> , <code>fine_status</code> (default Unpaid), <code>payment_date</code>	→ <code>borrow_transactions</code> (CASCADE)
<code>meeting_rooms</code>	<code>room_id</code> (varchar 10)	<code>room_name</code> , <code>capacity</code> , <code>location_floor</code> , <code>status</code> (default Available)	—
<code>room_reservations</code>	<code>room_reservation_id</code> (varchar 20)	<code>booker_name</code> , <code>booker_phone</code> , <code>booking_date</code> , <code>start_time</code> , <code>end_time</code> , <code>purpose</code> , <code>status</code> (default Confirmed)	→ <code>meeting_rooms</code> , → <code>members</code> (SET NULL)

ข้อสังเกต: `u5_lib.members` เป็นตารางที่เชื่อมทั้ง 3 ฝั่งเข้าด้วยกัน (นักศึกษาจาก `core`, อาจารย์/บุคลากรจาก `u6_hr`) — สมาชิกห้องสมุด 1 แถวควรมีค่าไม่ว่างเพียงคอลัมน์เดียวจาก 3 คอลัมน์ (`student_id` / `hr_faculty_member_id` / `hr_staff_id`) ตาม `member_type` แต่ในโครงสร้างปัจจุบัน**ไม่มี CHECK constraint บังคับ** ว่าต้องเลือกอย่างใดอย่างหนึ่งเท่านั้น (ทางทฤษฎีสามารถกรอกได้พร้อมกันหลายคอลัมน์ หรือไม่กรอกเลยก็ได้)

`room_reservations` มี CHECK constraint พิเศษ `chk_lib_max_3_hours` บังคับว่าจองห้องได้ครั้งละไม่เกิน 3 ชั่วโมง และ `end_time` ต้องมากกว่า `start_time` เสมอ

แผนภาพความสัมพันธ์ระดับ Schema



ข้อมูลตัวอย่างที่แทรกไว้ (Sample Data)

กลุ่ม	จำนวนแถวโดยประมาณ
core	1 สถาบัน, 3 คณะ, 6 สาขา, 6 วิชาเอก, 8 รายวิชา, 2 ภาคการศึกษา, 2 อาคาร, 4 ห้องเรียน, 10 นักศึกษา, 5 อาจารย์
u1_reg	3 เงื่อนไขวิชาบังคับก่อน, 6 กลุ่มเรียน, 10 คู่อาจารย์ที่ปรึกษา, 3 ประเภททุน, 3 การให้ทุน
u2_enroll	15 รายการลงทะเบียน
u3_grade	7 งานที่มอบหมาย, 17 คะแนนย่อย, 13 บันทึกการเข้าเรียน, 1 บันทึกแก้เกรด
u4_fee	3 ประเภทค่าธรรมเนียม, 5 ใบแจ้งหนี้, 8 รายการย่อยในใบแจ้งหนี้, 4 การชำระเงิน
u6_hr	4 ระดับตำแหน่งวิชาการ, 3 ประเภทการจ้าง, 5 ตำแหน่งงาน, 5 อาจารย์ (HR), 5 บุคลากรสายสนับสนุน, 3 หลักสูตร, 3 งบประมาณสาขา, 3 ผลงานตีพิมพ์, 3 ประเภทการลา
u5_lib	5 ประเภทสื่อ, 5 หมวดหมู่, 5 รายการทรัพยากร, 5 สำเนาจริง, 2 สื่อดิจิทัล, 8 สมาชิก, 5 รายการยืม-คืน, 3 การจอง, 1 ค่าปรับ, 3 ห้องประชุมกลุ่มย่อย, 3 การจองห้อง

สรุปประเด็นที่ควรตรวจสอบก่อนใช้งานจริง

- `u1_reg.advisors` — PK เป็น `student_id` เดี่ยว จำกัดให้มีอาจารย์ที่ปรึกษาได้คนเดียวตลอดการศึกษา ไม่รองรับการเปลี่ยนที่ปรึกษารายภาค
- Polymorphic** `owner_type / owner_id` ในตาราง 16 ตารางของ `u6_hr` — ไม่มี FK บังคับจริง ต้องเช็ค referential integrity ที่แอปพลิเคชัน
- `u6_hr.room_bookings.room_number` เป็นข้อความอิสระ ไม่ผูกกับ `core.rooms` ต่างจาก `u5_lib.room_reservations` ที่ผูกถูกต้อง
- `u5_lib.members` ไม่มี CHECK constraint บังคับว่าต้องกรอก `student_id/hr_faculty_member_id/hr_staff_id` เพียงคอลัมน์เดียวตาม `member_type`

5. `u4_fee.invoices.total_amount` และ `payments.amount_paid` ไม่ได้ผูกการคำนวณอัตโนมัติกับผลรวมของ `invoice_items` — ต้องดูแลความถูกต้องเองที่แอปพลิเคชัน
6. `u3_grade.grade_change_logs.changed_by` และ `u6_hr.performance_appraisals.evaluator_id` / `leave_approvals.approver_id` เป็นฟิลด์อิสระที่ไม่ได้ FK ไปหาอาจารย์/บุคลากรจริง